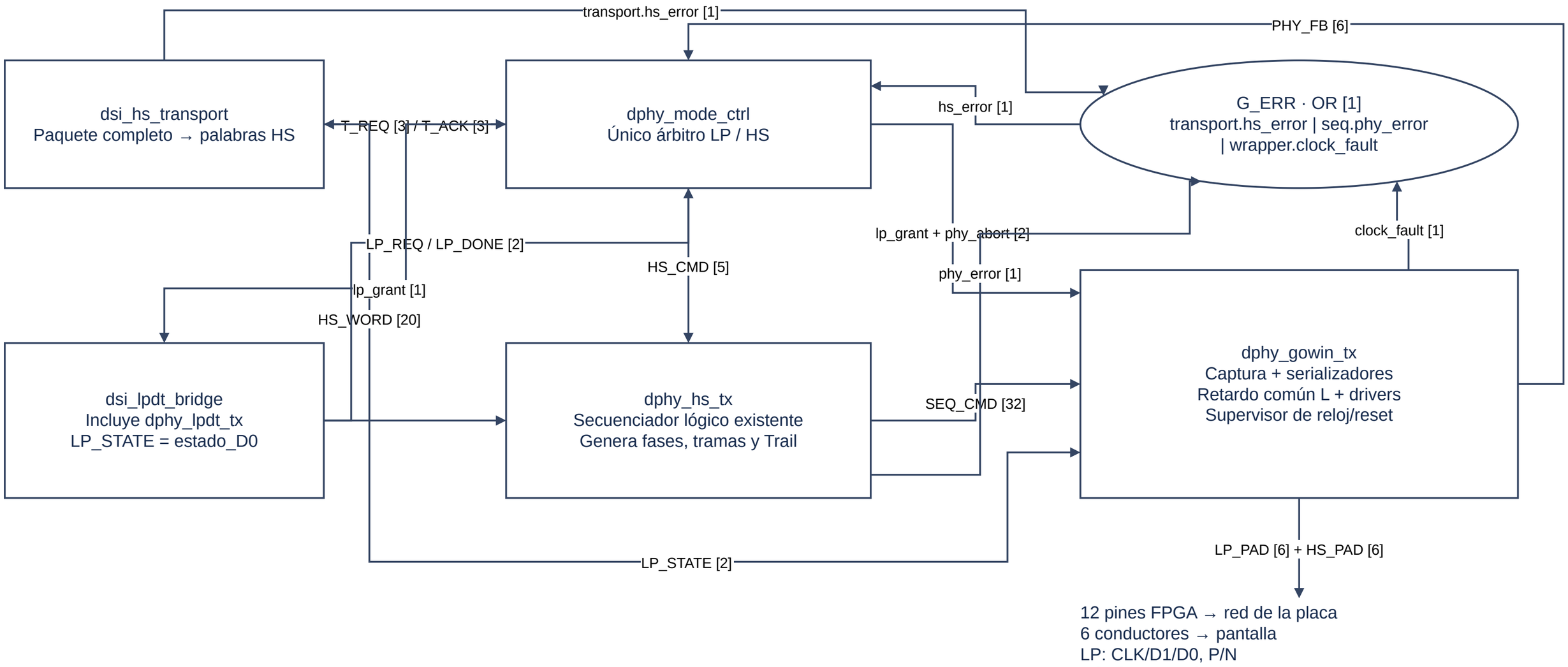
DPHY_GOWIN_TX · INTERCONEXIÓN ENTRE MÓDULOS

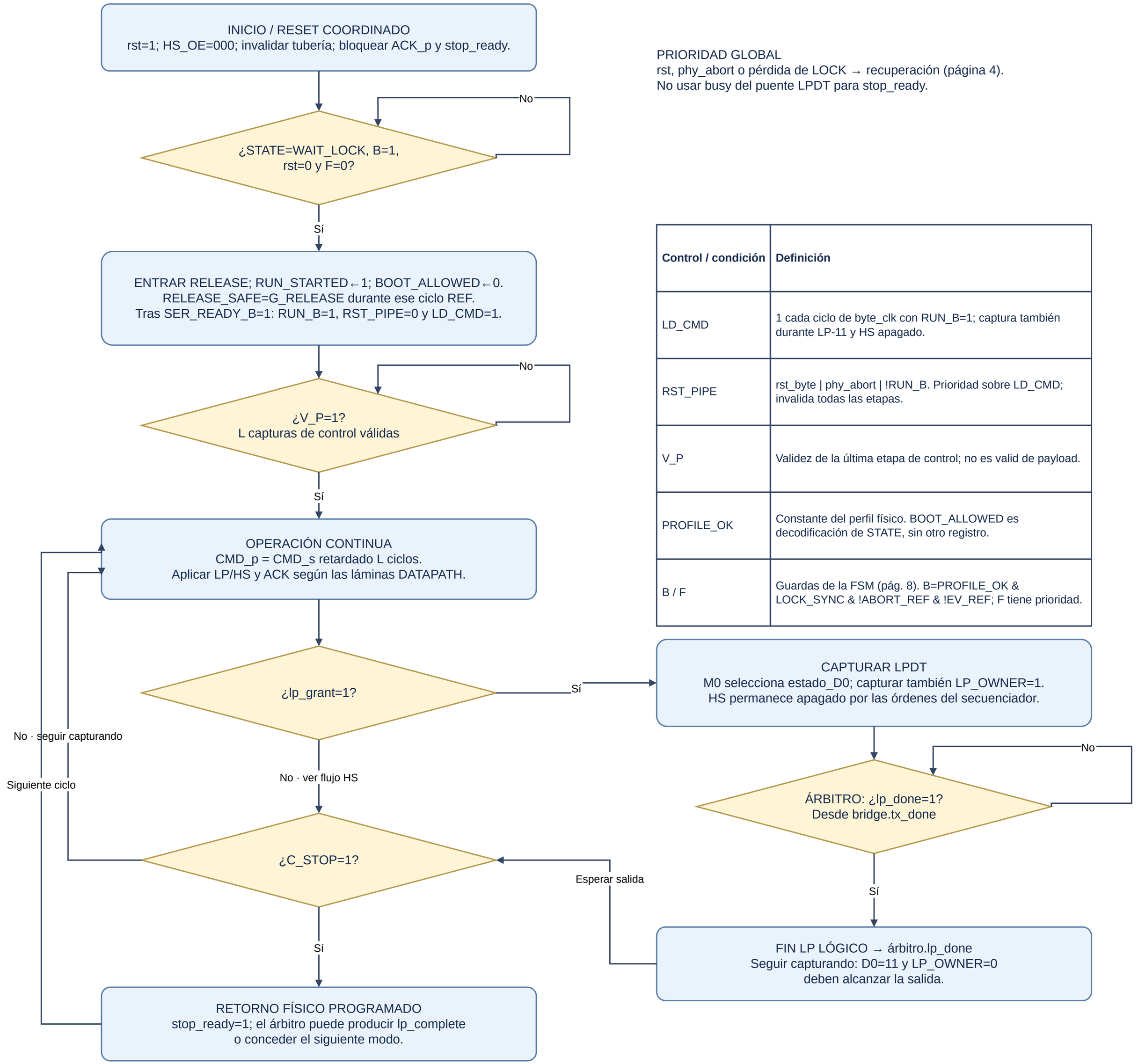
Las flechas identifican buses de la tabla inferior. _s = secuenciador lógico; _p = salida física programada / retorno al árbitro.



Bus	Campos y sentido exacto
HS_WORD [20]	transporte → secuenciador: hs_word_data[16], keep[2], valid, last. hs_grant viene del árbitro.
T_REQ [3] / T_ACK [3]	→ árbitro: hs_req, hs_done, hs_data_ready. Retorno: hs_req_ready, hs_grant, hs_complete. Error por G_ERR.
HS_CMD [5]	árbitro → secuenciador: clk_start_req, data_start_req, data_stop_req, clk_stop_req, hs_grant.
SEQ_CMD [32]	→ wrapper: frame[16], kind[2], HS_ON[2], toggle, LP[6], ACK_s[4], local_stop_s.
PHY_FB [6]	→ árbitro: ACK_p[4], stop_ready, pll_ready. clock_fault va por separado a G_ERR.
LP_REQ / LP_DONE	puente → árbitro: lp_req, tx_done → lp_done. lp_grant → permiso_arranque del puente y M0 del wrapper.
Errores (OR externo)	árbitro.hs_error = transporte.hs_error | secuenciador.phy_error | wrapper.clock_fault.
Reset y reloj	ref_clk=clk40 → wrapper; byte_clk + rst_byte → cadena común. rst es reset coordinado externo.
Salidas del árbitro	lp_complete / hs_complete → productores. El wrapper no genera esos pulsos ni recibe hs_word_last.

DPHY_GOWIN_TX · FLUJO GENERAL / LPDT

Una caja es una operación, no necesariamente un estado o un ciclo. Captura continua durante reposo, preparación y retorno LP.

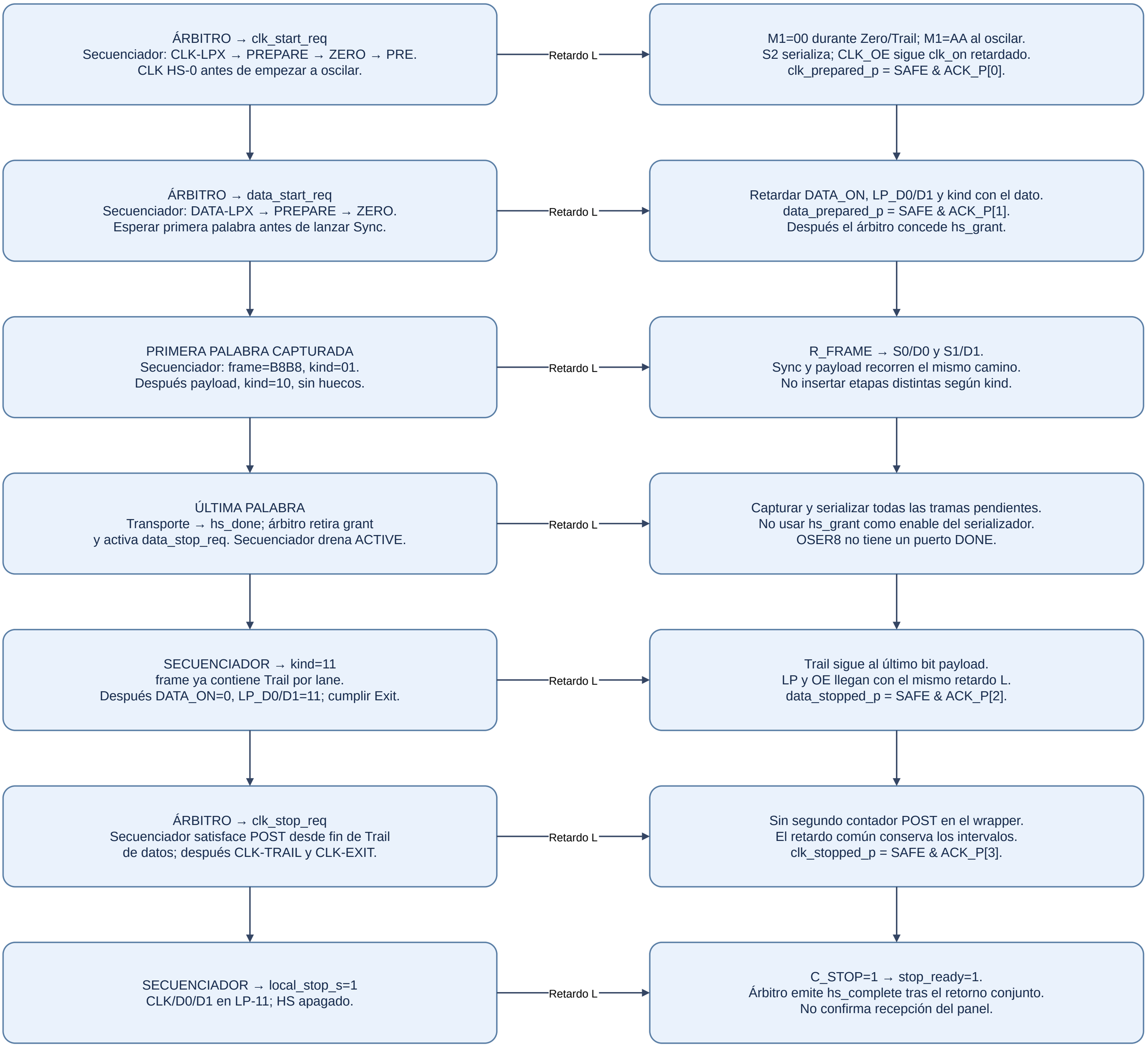


PRIORIDAD GLOBAL
rst, phy_abort o pérdida de LOCK → recuperación (página 4).
No usar busy del puente LPDT para stop_ready.

Control / condición	Definición
LD_CMD	1 cada ciclo de byte_clk con RUN_B=1; captura también durante LP-11 y HS apagado.
RST_PIPE	rst_byte | phy_abort | !RUN_B. Prioridad sobre LD_CMD; invalida todas las etapas.
V_P	Validez de la última etapa de control; no es valid de payload.
PROFILE_OK	Constante del perfil físico. BOOT_ALLOWED es decodificación de STATE, sin otro registro.
B / F	Guardas de la FSM (pág. 8). B=PROFILE_OK & LOCK_SYNC & !ABORT_REF & !EV_REF; F tiene prioridad.

DPHY_GOWIN_TX · FLUJO DE RÁFAGA HS

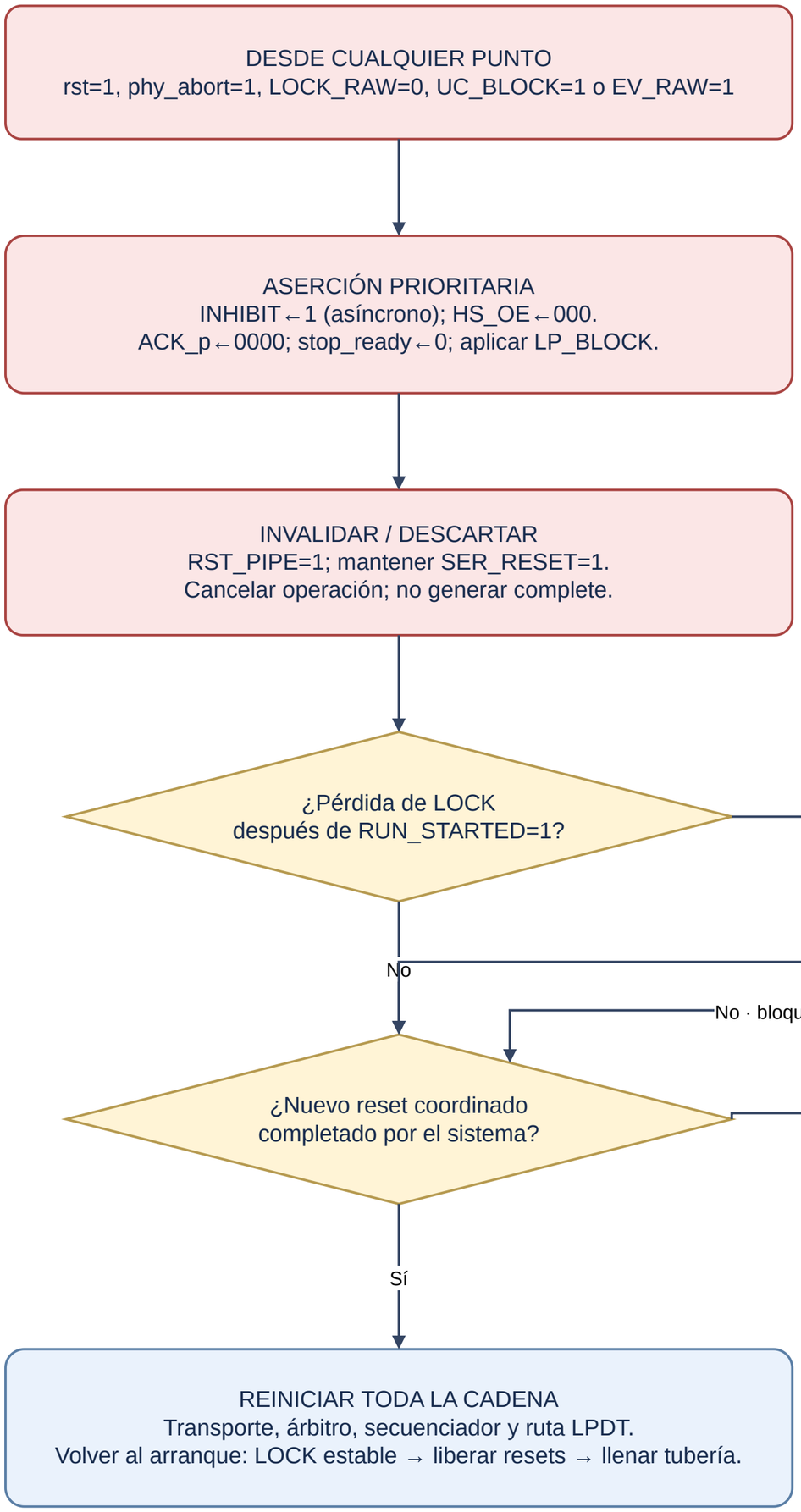
Izquierda: operaciones de dphy_hs_tx. Derecha: efecto del wrapper. Datos, LP/HS y confirmaciones se desplazan conjuntamente L ciclos.



keep=11 se comprueba en el secuenciador. El wrapper no añade relleno a paquetes impares.
Reset, phy_abort o pérdida de PLL → página 4. El perfil debe demostrar latencia común hasta los pads; de lo contrario esta arquitectura no es aplicable.

DPHY_GOWIN_TX · RESET, ABORTO Y PÉRDIDA DE PLL

La inhibición final no espera a FCLK. Recuperación por reset coordinado; clear_error del árbitro no recupera el wrapper.



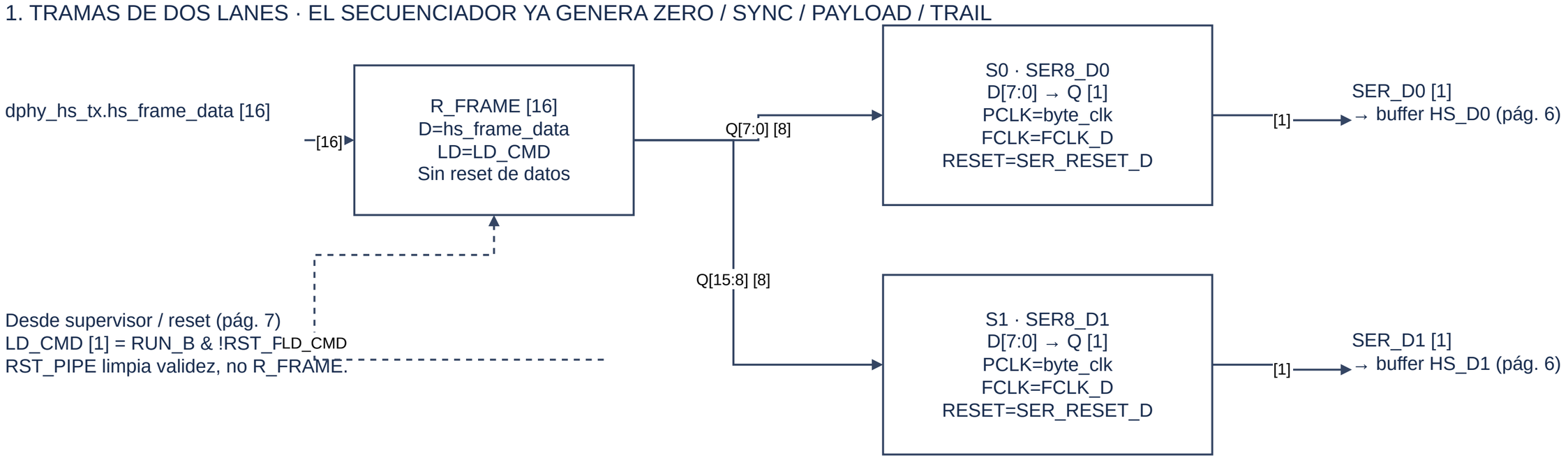
Prioridad	Efecto
rst	Limpiar CLOCK_FAULT y RUN_STARTED; mantener INHIBIT hasta arranque completo.
phy_abort	Inhibir HS e invalidar tubería. Exigir reset; no crea CLOCK_FAULT por sí solo.
!LOCK_RAW	Inhibir inmediatamente. Registrar CLOCK_FAULT si RUN_STARTED ya era 1.
RELEASE_SAFE	Solo supervisor tras reset y LOCK estable. La salida sigue bloqueada si V_P=0.

RAMAS LP DURANTE RECUPERACIÓN
LP_EN=P_LP_BLOCK_EN; LP_DAT=P_LP_BLOCK_DATA.
Modelo: EN=000000 (alta impedancia).
No forzar LP-11 contra un motor activo.
El perfil eléctrico final requiere validación de la placa.

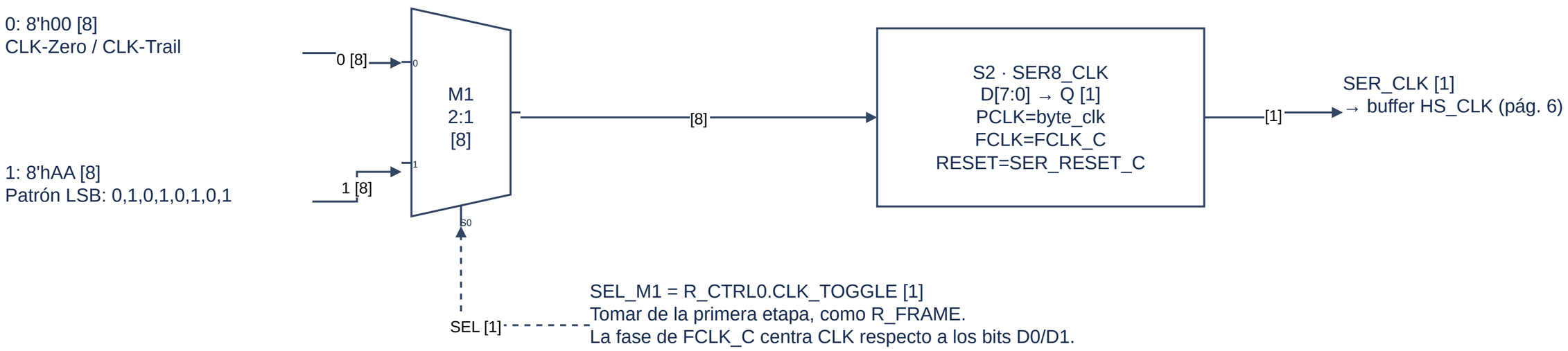
DPHY_GOWIN_TX · DATAPATH 1/3 · CAPTURA Y SERIALIZACIÓN

Bloques sin relleno · MUX con entradas numeradas · buses [n] · controles discontinuos · todos los registros ordinarios en ↑byte_clk.

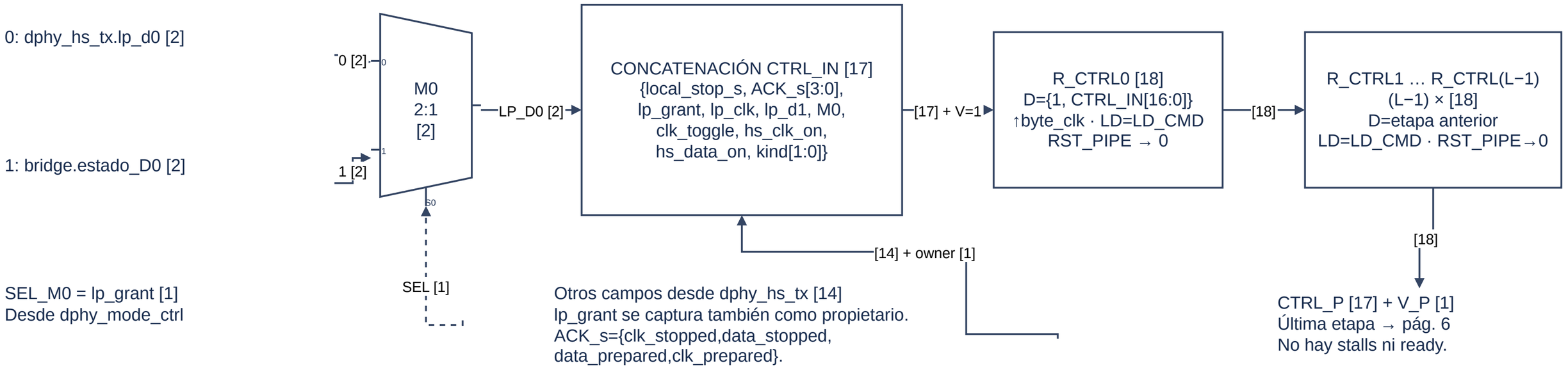
1. TRAMAS DE DOS LANES · EL SECUENCIADOR YA GENERA ZERO / SYNC / PAYLOAD / TRAIL



2. RELOJ DSI · SERIALIZAR UN PATRÓN; NO CORTAR FCLK CON UNA PUERTA LÓGICA



3. CONTROL CAPTURADO CON LA TRAMA · LP, PROPIEDAD Y ACK SIGUEN LA MISMA LATENCIA TOTAL L



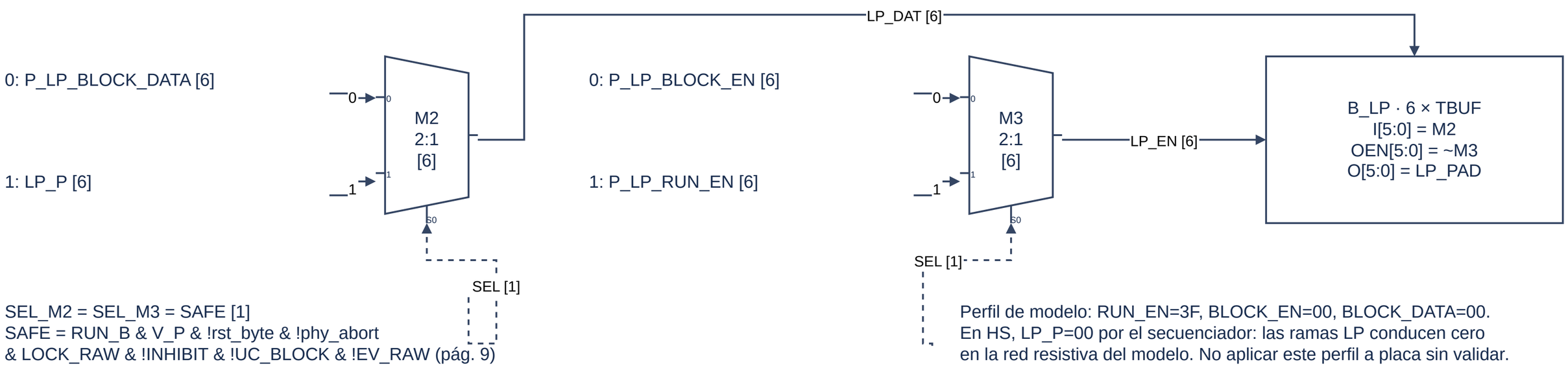
CONTRATO: L = 2 + J, J≥0, entero y común a S0/S1/S2. Las etapas internas de SER8 suman J.
En ↑k: R_FRAME←FRAME[k−1]; SER8 carga R_FRAME anterior; bit serial de FRAME[k−L]. CTRL_P[k]=CTRL_IN[k−L].
Perfil de dibujo/modelo: J=0, L=2. El valor físico debe caracterizarse. El adaptador MAP_BITS emite siempre el bit 0 primero.

DPHY_GOWIN_TX · DATAPATH 2/3 · SALIDAS Y CONDICIONES

Control registrado en la última etapa de la página 5. MUX y operadores separados. Todas las habilitaciones internas son activas altas.

CTRL_P [17] + V_P [1]: KIND=[1:0]; DATA_ON=[2]; CLK_ON=[3]; TOGGLE=[4]; LP={CLK,D1,D0}=[10:5];
OWNER=[11]; ACK={clk_stopped,data_stopped,data_prepared,clk_prepared}=[15:12]; LOCAL_STOP=[16].
Cada par LP={P,N}. Buses de salida LP_PAD / HS_PAD [6], en orden {CLK_P,CLK_N,D1_P,D1_N,D0_P,D0_N}.

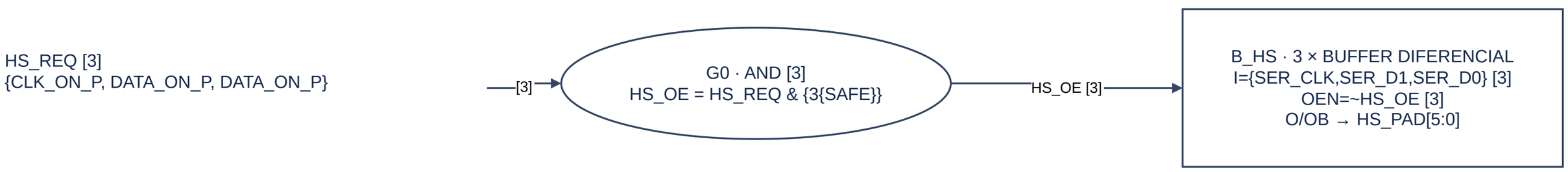
1. RAMAS LP · DOS MUX Y SEIS BUFFERS



SEL_M2 = SEL_M3 = SAFE [1]
SAFE = RUN_B & V_P & !rst_byte & !phy_abort
& LOCK_RAW & !INHIBIT & !UC_BLOCK & !EV_RAW (pág. 9)

Perfil de modelo: RUN_EN=3F, BLOCK_EN=00, BLOCK_DATA=00.
En HS, LP_P=00 por el secuenciador: las ramas LP conducen cero
en la red resistiva del modelo. No aplicar este perfil a placa sin validar.

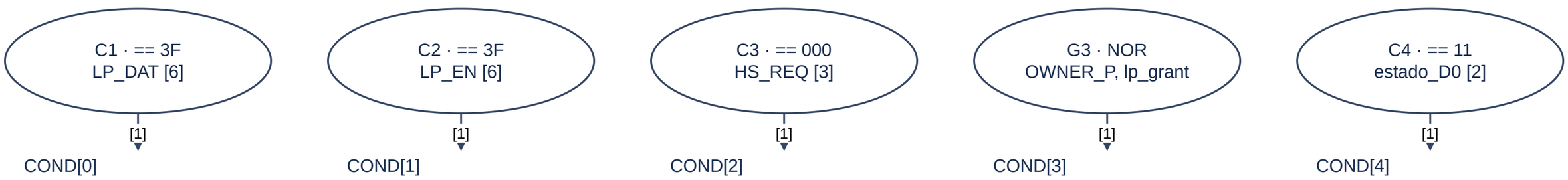
2. RAMAS HS · NO USAR hs_grant PARA CORTAR EL DRENAJE



3. ACK · CUATRO CONFIRMACIONES HACIA dphy_mode_ctrl



4. REPOSO CONJUNTO · COMPARADORES Y NOR SIN MEMORIA ADICIONAL

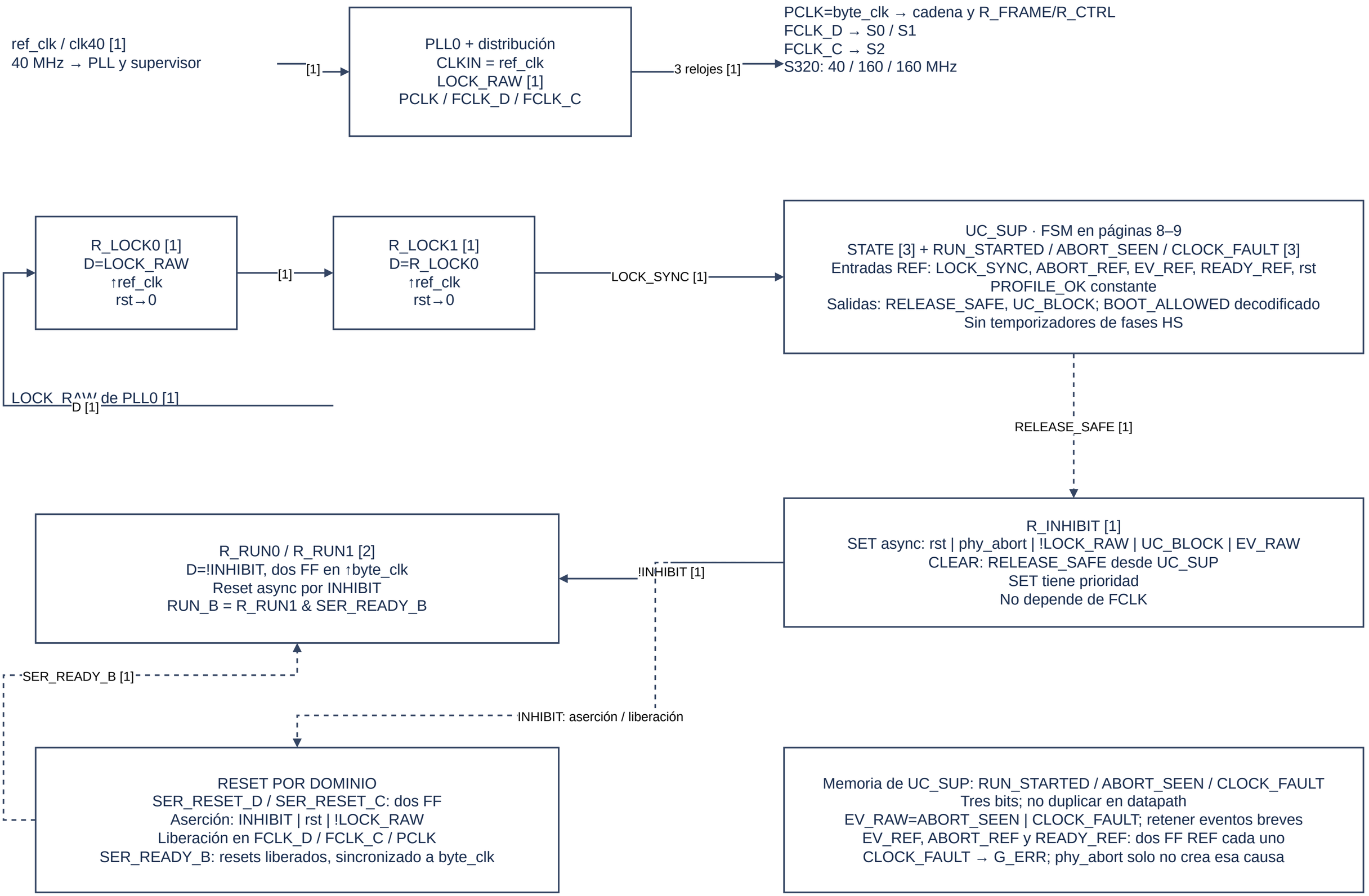


COND[4:0]={C4,G3,C3,C2,C1}
SAFE [1], LOCAL_STOP_P [1]



DPHY_GOWIN_TX · DATAPATH 3/3 · RELOJES, RESET Y SUPERVISIÓN

Relojes relacionados para toda la cadena. REF_CLK permanece independiente del PLL. Aserción asíncrona y liberación sincronizada de los resets físicos.

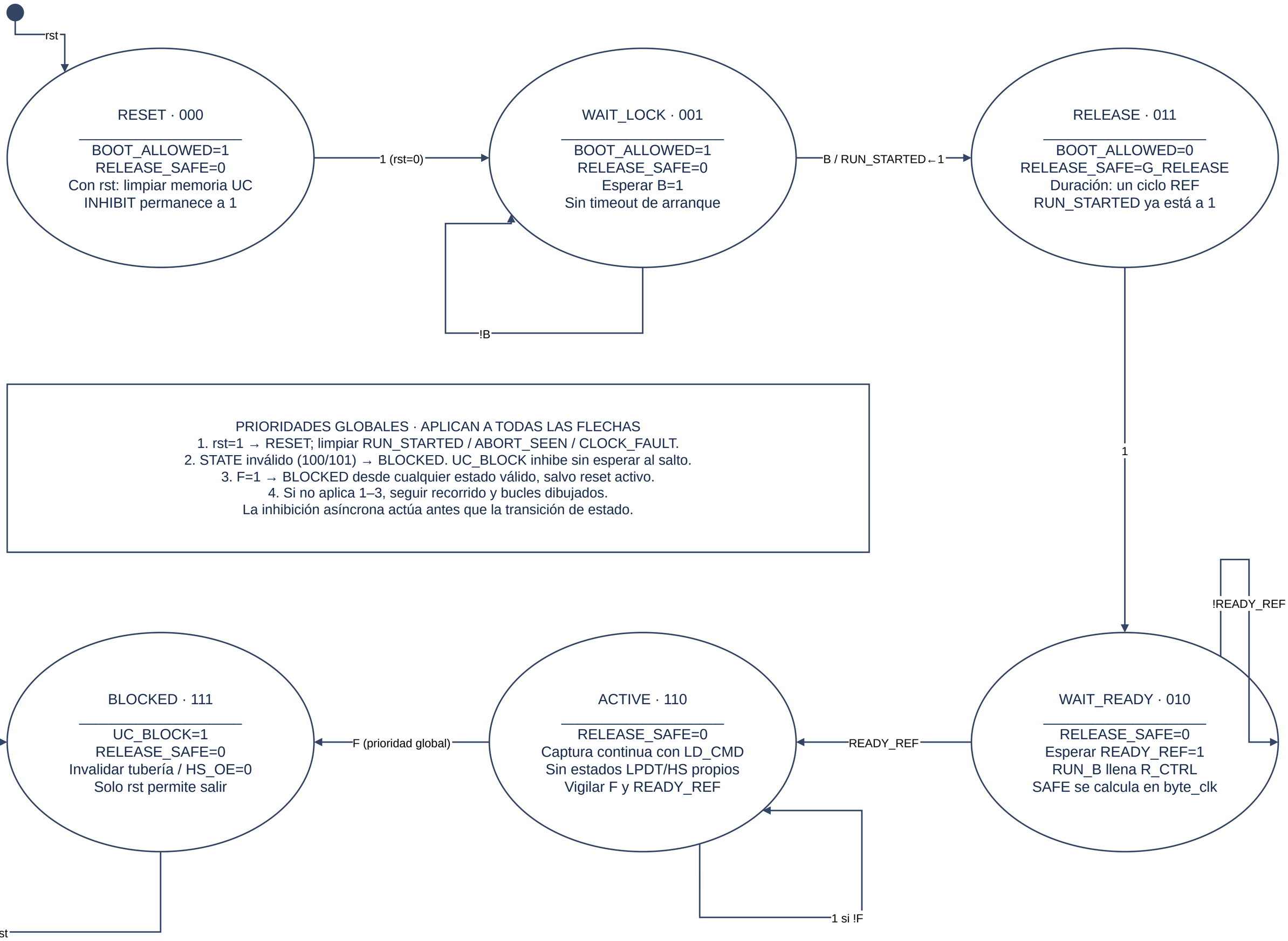


RST_BYTE [1] → reset coordinado del dominio de la cadena
Aserción async: rst | !LOCK_RAW; liberación con 2 FF en byte_clk.
PHY_ABORT no se conecta a RST_BYTE: evita un rearme automático del árbitro.

Salida / control	Destino y ecuación
LD_CMD / RST_PIPE	Registros pág. 5: LD_CMD=RUN_B & !RST_PIPE; RST_PIPE=rst_byte | phy_abort | !RUN_B.
SAFE	M2/M3 y G0/G1/C_STOP pág. 6. Incluye LOCK_RAW e INHIBIT; inhibe aun sin FCLK.
pll_ready	Árbitro.pll_locked = RUN_B & !INHIBIT & LOCK_RAW & !rst_byte & !UC_BLOCK & !EV_RAW.
clock_fault	OR externo con transporte.hs_error y secuenciador.phy_error → árbitro.hs_error.

DPHY_GOWIN_TX · FSM 1/2 · SUPERVISOR UC_SUP

STATE [3] en ↑ref_clk (40 MHz). Entradas evaluadas antes del flanco. Salidas combinacionales con valores por defecto. Sin next_state ni ACTION.



Símbolo	Definición exacta (1 bit)
B	PROFILE_OK & LOCK_SYNC & !ABORT_REF & !EV_REF
EV_RAW / EV_REF	ABORT_SEEN | CLOCK_FAULT / su copia sincronizada con dos FF REF
READY_REF	Dos FF REF de (RUN_B & V_P & !rst_byte); no cruza el bus de tramas
F	EV_REF | (RUN_STARTED & (!LOCK_SYNC | ABORT_REF | !PROFILE_OK)) | (STATE=ACTIVE & !READY_REF)
G_RELEASE	!rst & PROFILE_OK & LOCK_SYNC & LOCK_RAW & !phy_abort & !EV_RAW

Codificación Gray del recorrido normal: 000→001→011→010→110. Evita atravesar códigos inválidos al avanzar entre fases normales.
Las flechas normales presuponen !rst, STATE válido y !F. BLOCKED no sale por clear_error, recuperación de LOCK o retirada de phy_abort.

DPHY_GOWIN_TX · FSM 2/2 · CONTROLES, MEMORIA Y CRUCES

UC: STATE [3] + RUN_STARTED [1] + ABORT_SEEN [1] + CLOCK_FAULT [1] = 6 bits. BOOT_ALLOWED es decodificación, no otro registro.

STATE	BOOT_ALLOWED	UC_BLOCK	RELEASE_SAFE	Acción / condición de salida
RESET 000	1	0	0	Con rst limpiar memoria. !rst & !F → WAIT_LOCK.
WAIT_LOCK 001	1	0	0	B & !F: RUN_STARTED←1 y STATE←RELEASE en ese ↑REF.
RELEASE 011	0	0	G_RELEASE	Limpiar R_INHIBIT solo si G_RELEASE=1. !F → WAIT_READY.
WAIT_READY 010	0	0	0	READY_REF & !F → ACTIVE. Captura ya funciona cuando RUN_B=1.
ACTIVE 110	0	0	0	Mantener operación; cualquier F → BLOCKED.
BLOCKED 111	0	1	0	Mantener bloqueo hasta rst, aun con LOCK y phy_abort recuperados.
Inválido 100/101	0	1	0	Bloquear inmediatamente; en siguiente ↑REF ir a BLOCKED.

CAPTURA PERSISTENTE DE EVENTOS · rst TIENE PRIORIDAD

RUN_STARTED [1]
↑ref_clk
rst → 0
BOOT_FIRE → 1
Resto: conservar

ABORT_SEEN [1]
Set asíncrono:
RUN_STARTED & phy_abort
rst → 0; resto conservar
No se limpia al retirar phy_abort

CLOCK_FAULT [1]
Set asíncrono:
RUN_STARTED & !LOCK_RAW
rst → 0; resto conservar
Salida clock_fault → G_ERR

BOOT_FIRE = !rst & (STATE=WAIT_LOCK) & B & !F
Evento de aceptación anterior al ↑REF. Estado RELEASE y RUN_STARTED=1 aparecen juntos después del flanco.
En RELEASE, un evento breve queda en EV_RAW e impide volver a limpiar INHIBIT, aunque el nivel de entrada ya se haya recuperado.

R_INHIBIT [1] · MEMORIA DE SEGURIDAD
SET async = rst | phy_abort | !LOCK_RAW | UC_BLOCK | EV_RAW
CLEAR = RELEASE_SAFE; SET > CLEAR > conservar
No depende de FCLK. No borra una causa retenida.

CRUCES HACIA REF_CLK · SOLO SEÑALES DE UN BIT

Origen	Sincronizador / destino	Uso
LOCK_RAW	R_LOCK0/R_LOCK1 en REF (existentes)	LOCK_SYNC → B/F; no sustituye la inhibición raw.
phy_abort	R_ABORT0/R_ABORT1 en REF (2 FF)	ABORT_REF → B/F; startup puede esperar al reset del árbitro.
ABORT_SEEN | CLOCK_FAULT	R_EV0/R_EV1 en REF (2 FF)	EV_REF → F; EV_RAW inhibe directamente sin esperar CDC.
RUN_B & V_P & !rst_byte	R_READY0/R_READY1 en REF (2 FF)	READY_REF → transición WAIT_READY/ACTIVE; solo observación.

SAFE = RUN_B & V_P & !rst_byte & !phy_abort & LOCK_RAW & !INHIBIT & !UC_BLOCK & !EV_RAW
pll_ready = RUN_B & !rst_byte & LOCK_RAW & !INHIBIT & !UC_BLOCK & !EV_RAW. ACK_OUT=ACK_P & {4{SAFE}}; stop_ready=C_STOP.
RST_PIPE y LD_CMD conservan las ecuaciones del datapath. ACTIVE no se añade como permiso: hacerlo alteraría el retardo L.